

No Cascading Style Sheets concepts

1. No default Cascading. Exclude HTML hierarchy
2. Element Type checks. By Id?
 - a. `div` only `{}`
 - b. `#elemId*` `{}`
3. Variable types
4. Additional DataTypes
 - a. Enums `button-colors: {custom-red, custom-green, custom-blue}`, usage - `color: custom-red`; `color: any-other` - results in error;
 - b. style definition types. Also possible to make params optional with pre defaults
style `StyleName (param1: Color, param2?: Number = 2): <elementId | HTMLInputElement>{`
 `color: custom-color`
 `font: Font (default types)`
 `Line-spacing: param2`
 `}`

 `ButtonStyle: StyleName {`
 `color: Custom-Green;`
 `font: "New Times Roman";`
 `???`
 `}`
5. Parameterized classes.
 `.ClassName (param1: Color, param2: Number, param3: Style): <elementId | HTMLInputElement>{`
 `color: param1`
 `font:"Times New Roman"`
 `Line-spacing: param2`
 `div : param3`
 `}`
 You can pass whole styles for sub elements, useful for headless components implementation as well.
6. Style definition type used to describe possible styling for components, but without defining style itself. Useful for headless components
 style `ButtonDef (param1: red | blue): div {`
 `color: param1`
 `Line-spacing: param2`
 `}`

// Depends on library like react etc.

Button = `<div style="extends ButtonDef">Press Me</div>`

// Library usage

```
<Button style="{color: blue}" />
```

OR

```
(param1: Color) implements ButtonDef {  
    color: param1  
    font:"Times New Roman"  
    Line-spacing: param2  
}  
<Button style="StyledButton">
```